

# Terraform Project

---

신재섭 (조장)  
전병욱  
이은석  
정태환



# 목차

|     |              |     |           |     |                   |     |                 |
|-----|--------------|-----|-----------|-----|-------------------|-----|-----------------|
| 1   | Terraform 정의 | 2   | 환경 구축     | 3   | Terraform 기본 개념   | 4   | Terraform 작동 원리 |
| 1-1 | IaC          | 2-1 | Terraform | 3-1 | provider          | 4-1 | init            |
| 1-2 | Terraform    | 2-2 | AWS CLI   | 3-2 | resource          | 4-2 | validate        |
| 1-3 | 프로비저닝        | 2-3 | VSCode    | 3-3 | variable          | 4-3 | plan            |
| 1-4 | CI/CD        |     |           | 3-4 | output            | 4-4 | apply           |
| 1-5 | 장점           |     |           | 3-5 | terraform.tfstate | 4-5 | destroy         |

# 목차

## 5 TF 파일 분석

5-1 구조 및 구성

5-2 Code

## 6 Github, Terraform Cloud, AWS 연동

6-1 Github repository 생성

6-2 Terraform Cloud 초기 구성 & Github 연동

6-3 Terraform Cloud & AWS 연동

## 7 Route 53 도메인 등록

7-1 도메인 확인

7-2 NS 레코드 확인 및 입력

7-3 출력 결과 확인

# 1. Terraform 정의

---



## 1-1. IaC

IaC (Infrastructure as Code)

인프라를 코드로 관리하는 방식

대표적인 IaC 도구

- Terraform
- Ansible
- AWS CloudFormation

## 1-2. Terraform

Terraform

- Terraform은 HashiCorp가 개발한 IaC 도구
- 코드로 인프라를 정의
- 여러 클라우드 및 온프레미스 환경에 일관되게 배포 가능



## 1-3. 프로비저닝

### 프로비저닝

필요한 IT 인프라 자원을 생성 및 설정 과정

### 프로비저닝 예

- 서버(EC2) 생성
- 네트워크(VPC) 구성
- 보안 그룹 설정
- 데이터베이스(RDS) 생성

## 1-4. CI/CD

### CI/CD

소프트웨어를 자동으로 빌드, 테스트, 배포하는  
개발 자동화 방식

### CI (Continuous Integration)

개발자가 코드를 수정 후 자동으로 빌드 및 테스트 수행  
→ 오류를 빠르게 확인 가능

### CD (Continuous Delivery / Deployment)

테스트가 완료된 코드를 서버에 자동 배포하는 과정

# 1-5. 장점

## Terraform의 장점

### 1. 자동화 가능

인프라를 코드로 작성하여 서버, 네트워크, DB 등을 자동으로 생성할 수 있다.

반복 작업을 줄이고 구축 시간을 단축할 수 있다.

### 2. 동일한 환경 재현 가능

같은 .tf 코드를 사용하면 언제 어디서든 동일한 인프라 환경을 만들 수 있다.

개발 환경과 운영 환경의 차이를 줄일 수 있다.

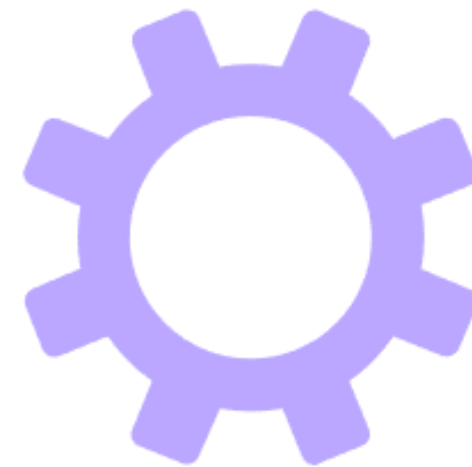
### 3. 버전 관리 및 협업 가능

Terraform 코드는 Git으로 관리할 수 있어 변경 이력을 추적할 수 있다.

여러 사람이 동시에 인프라를 관리하기에도 용이하다.

## 2. 환경 구축

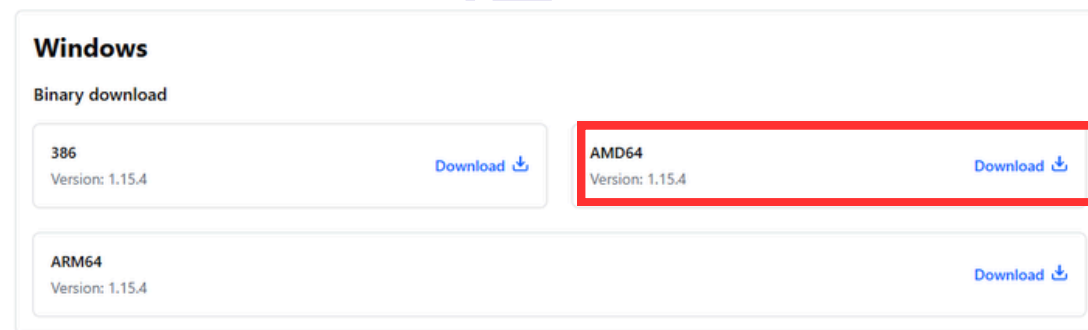
---





# 2-1. Terraform 설치

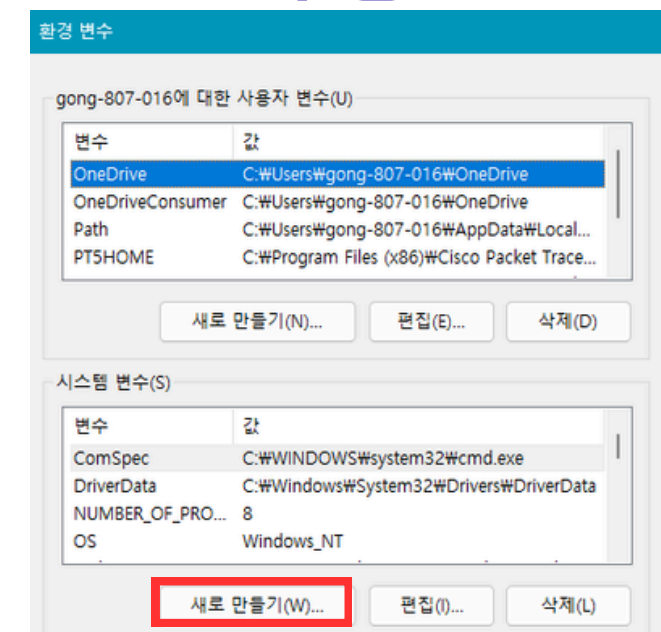
## 1. Terraform 다운로드



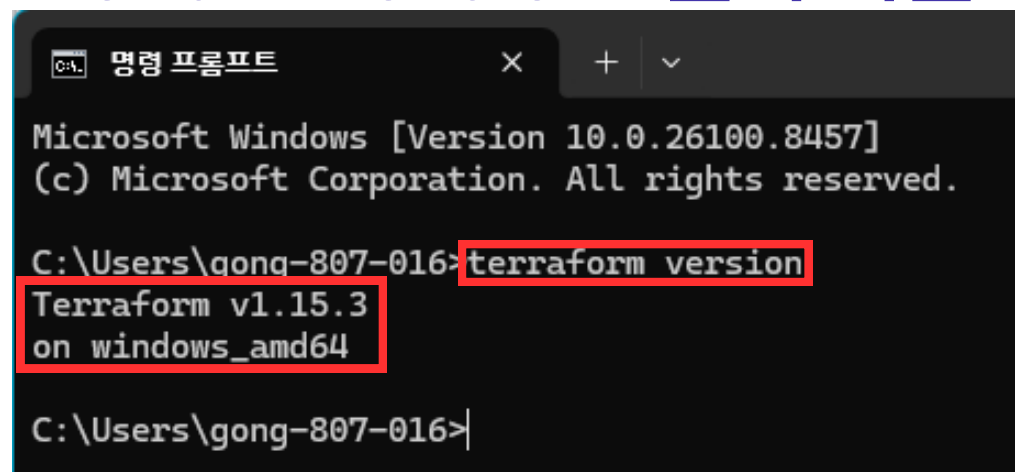
## 2. 환경 변수 설정



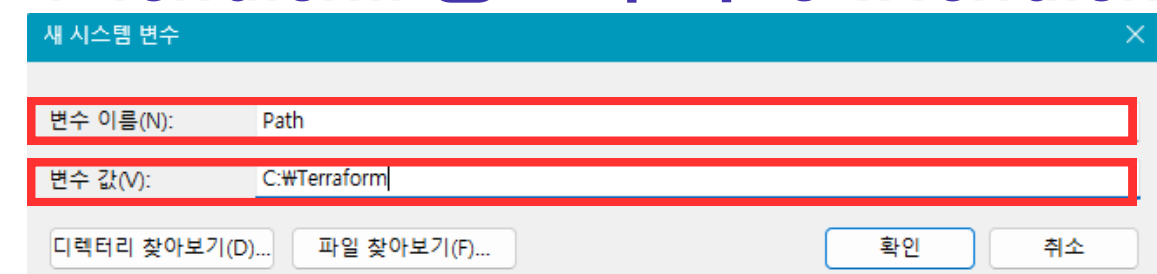
## 3. Path 수정



## 5. cmd → Terraform 설치 확인



## 4. Terraform 경로 추가 (C:\Terraform)



# 2-2. AWS CLI 연동

## 1. 파일 다운로드

### AWS CLI 설치 또는 업데이트

Windows에서 AWS CLI의 현재 설치를 업데이트하려면 업데이트할 때마다 새 설치 관리자를 다운로드 출시된 시기를 확인하려면 [GitHub에서 AWS CLI 버전 2 변경 로그](#)를 참조하세요.

1. Windows용 AWS CLI MSI 설치 관리자(64비트)를 다운로드하여 실행합니다.

<https://awscli.amazonaws.com/AWSCLIV2.msi>

또는 `msiexec` 명령을 실행하여 MSI 설치 관리자를 실행할 수 있습니다.

```
C:\> msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi
```

## 2. AWS 버전 확인

```
명령 프롬프트
Microsoft Windows [Version 10.0.26100.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\gong-807-016>aws --version
aws-cli/2.34.23 Python/3.14.3 Windows/x64
```

## 3. 액세스 키 생성

### 액세스 키 (1)

액세스 키를 사용하여 AWS CLI, AWS Tools for PowerShell, AWS SDK 또는 직접 AWS API 호출을 통해 AWS에 프로그래밍 방식 호출을 전송합니다. 한 번에 최대 두 개의 액세스 키(활성 또는 비활성)를 가질 수 있습니다. [자세히 알아보기](#)

액세스 키 만들기

## 5. AWS 로그인 확인

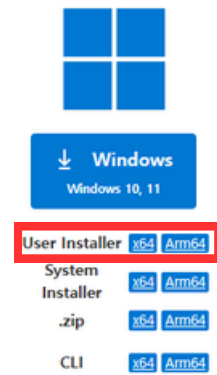
```
명령 프롬프트
C:\Users\gong-807-016>aws configure list
NAME : VALUE : TYPE : LOCATION
profile : <not set> : None : None
access_key : *****BRRJ : shared-credentials-file :
secret_key : *****5elw : shared-credentials-file :
region : ap-northeast-2 : config-file : ~/.aws/config
```

## 4. AWS Configure → 액세스 키 입력

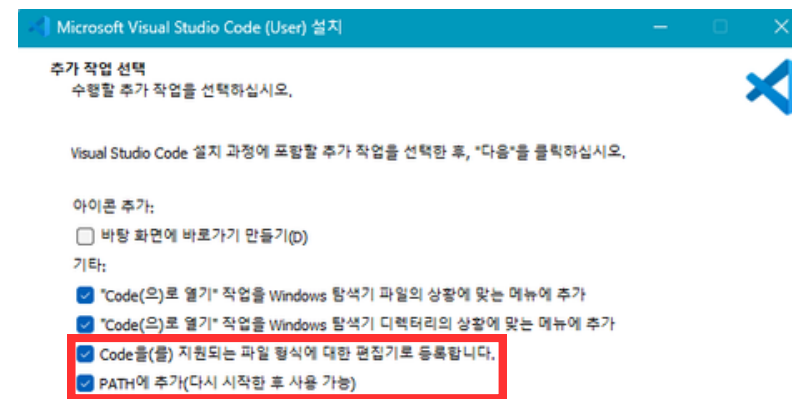
```
명령 프롬프트
C:\Users\gong-807-016>aws configure
AWS Access Key ID [*****BRRJ]: AK
AWS Secret Access Key [*****5elw]:
Default region name [ap-northeast-2]:
Default output format [json]:
```

## 2-3. VSCode 설치

### 1. 파일 다운로드

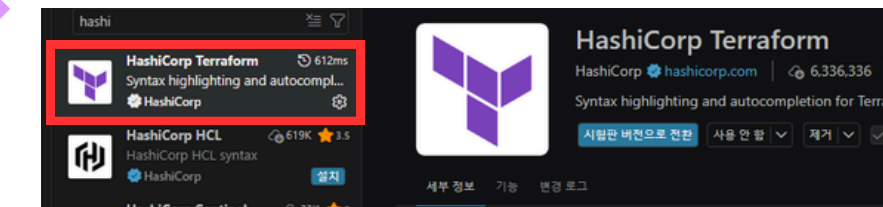


### 2. 체크박스 확인

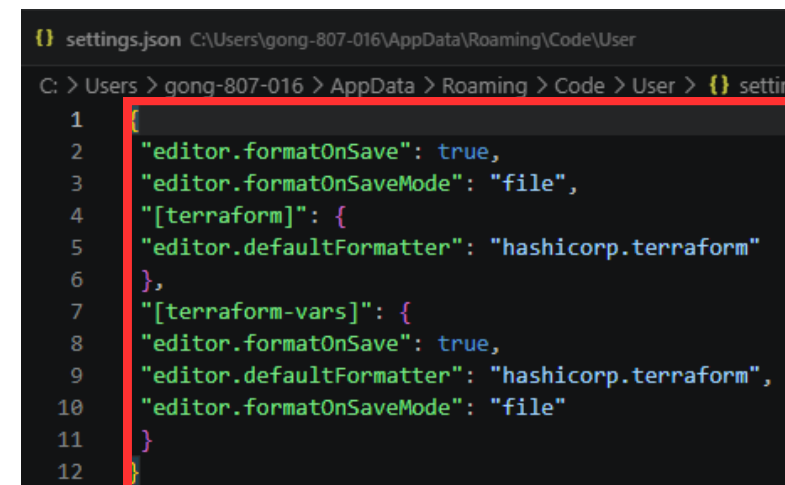


### 3. VSCode LH

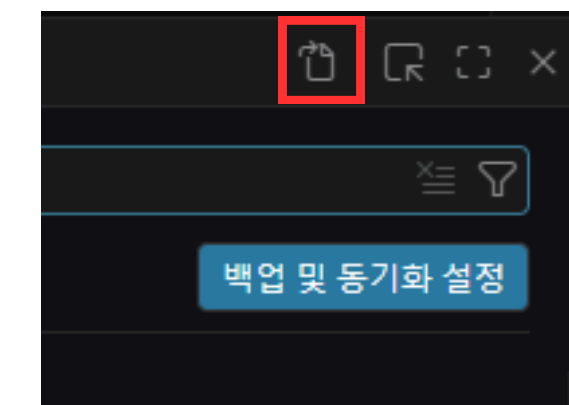
### HashiCorp Terraform 설치



### 5. JSON 내용 입력



### 4. 자동 포맷 구성



# 3. Terraform 기본 개념

---



# 3-1. Provider

## Provider

Terraform이 클라우드 서비스와  
연결할 수 있도록 해주는 플러그인

## Provider 기본 구조

```
provider "<PROVIDER_NAME>" {  
  # Configuration arguments  
}
```

## Provider 역할

### 1. 인증 정보 제공

테라폼이 해당 클라우드 계정에 접근할 수 있도록  
인증 정보를 설정

### 2. 기본 설정 지정

해당 클라우드의 리소스가 생성될 기본 리전과 같은  
전역 적인 설정을 지정



## 3-2. Resource

### Resource

Terraform에서 실제 생성되는 인프라 자원

### Resource 기본 구조

```
resource "<RESOURCE_TYPE>" "<LOCAL_NAME>" {  
  # Configuration arguments  
}
```

### Resource 블록의 구조

resource

블록의 타입

RESOURCE\_TYPE

프로바이더가 제공하는 리소스의 종류

LOCAL\_NAME

구성 파일 내에서 해당 리소스를 참조할 때 사용하는 고유이름

# Configure arguments

리소스의 속성 정의

## 3-3. Variable

### Variable

Terraform 코드에서 값을 변수처럼  
저장하고 재사용하기 위한 기능

### Variable 기본 구조

```
variable "[VARIABLE_NAME]" {  
  type = type_constraint  
  default = default_value  
  description = description_string  
  ...  
}
```

### Variable 블록의 구조

#### variable

블록의 타입

#### VARIABLE\_NAME

구성 파일 내에서 변수를 참조할 때 사용되는 이름

#### default

변수에 값이 제공되지 않을 경우 사용될 기본 값 지정

#### type

변수가 받아야 할 자료형 제약 조건을 지정

#### description

변수의 용도를 설명

# 3-4. Output

## Output

Terraform 실행 후 생성된 결과 값을 출력하는 기능

## Output 기본 구조

```
resource "[OUTPUT_NAME]" {  
  value = expression  
  description = description_string  
  ...  
}
```

## Output 블록의 구조

output

블록의 타입

OUTPUT\_TYPE

출력될 값에 대한 고유한 이름

value

필수 인자, 출력할 실제 값 또는 그 값을 계산하는 표현식

description

출력될 값이 무엇인지 설명

sensitive

값을 출력하라 때 터미널이나 원격 상태에서 숨김처리 선택 지정



## 3-5. Terraform.tfstate 파일

### 테라폼 상태 파일 (.tfstate)

Terraform이 현재 인프라 상태를 저장하는 파일

### 테라폼 상태 파일 역할

#### 실제 인프라 매칭

상태 파일은 테라폼 구성코드와 실제 클라우드 리소스를 연결하는 다리 역할.

파일에는 리소스의 ID, 속성 값, 메타데이터 등 모든 정보가 저장되어 있음.

#### 성능 최적화

terraform plan이나 apply를 실행할 때, 매번 클라우드 API를 호출하여 모든 리소스를 조회하는 대신,

상태 파일의 정보를 먼저 참고하여 변경 사항을 효율적으로 감지하고 실행 시간을 단축

#### 의존성 관리

리소스 간의 관계와 의존성을 추적하여, 올바른 순서로 리소스를 생성하거나 파괴할 수 있게 보장

# 4. Terraform 작동 원리

---



## 4-1. Init

### terraform init

Terraform 작업을 시작하기 위해  
필요한 초기 설정을 수행하는 명령어

### 특징

Terraform 프로젝트 시작 시 가장 먼저 실행  
.terraform 디렉토리 생성  
필요한 Provider 자동 설치

## 4-2. Validate

### terraform validate

Terraform 코드의 문법과 설정이  
올바른지 검사하는 명령어

### 특징

인프라 생성 없이 검사만 수행  
코드 작성 후 오류 확인 가능  
배포 전 검증 단계에서 자주 사용

## 4-3. Plan

### terraform plan

Terraform이 실제로 어떤 변경 작업을 수행할지  
미리 보여주는 명령어

### 특징

실제 적용은 하지 않음  
변경 내용을 사전에 확인 가능  
실수 방지 및 검토에 유용

## 4-4. Apply

### terraform apply

Terraform 코드 내용을  
실제 클라우드 환경에 적용하는 명령어

### 특징

실제 인프라 변경 수행  
terraform.tfstate 파일 업데이트  
배포 단계에서 사용

# 4-5. Destroy

## terraform destroy

Terraform이 생성한 인프라 자원을 삭제하는 명령어

### 특징

Terraform이 생성한 자원 전체 삭제 가능

테스트 환경 정리에 자주 사용

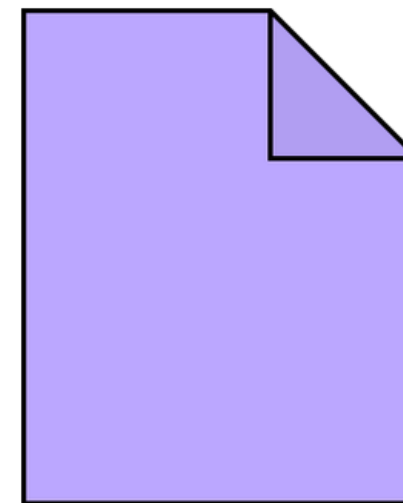
비용 절감 유용





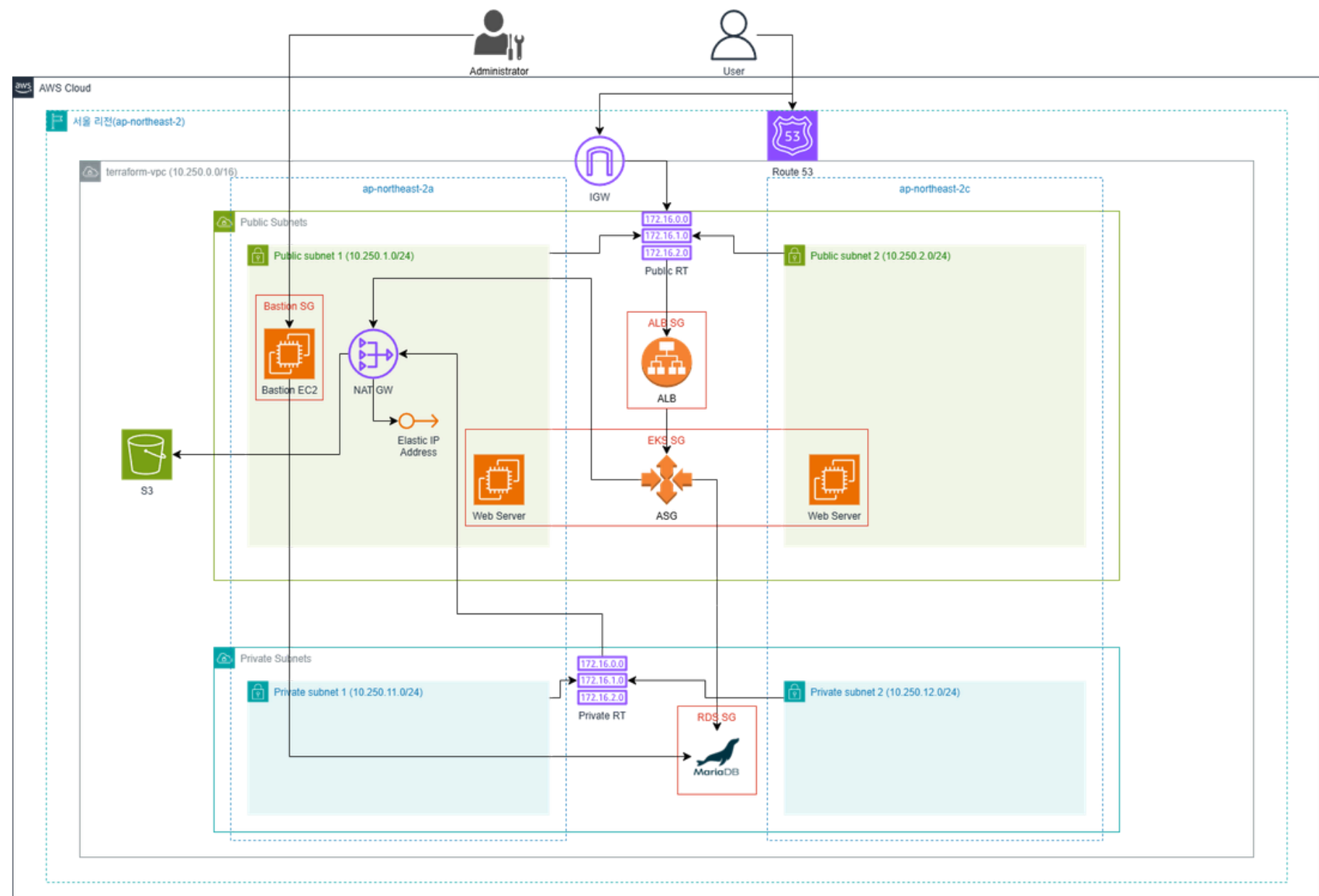
# 5. TF 파일 분석

---



# 5-1. 구조 및 구성

## 구성도



# 1. Terraform Provider.tf

```
terraform {  
  required_version = ">= 1.0.0, <2.0.0"  
  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "ap-northeast-2" #Asia Pacific (seoul) region  
}
```

테라폼 버전

1.0.0 이상 2.0.0 미만

프로바이더

aws

플러그인 버전

5.0 이상 6.0 미만

가용 영역

ap-northeast-2 #Asia Pacific (seoul) region



# 2. VPC.tf

```
resource "aws_vpc" "terraform-vpc" {  
  cidr_block      = "10.250.0.0/16"  
  enable_dns_support = true  
  enable_dns_hostnames = true  
  tags = {  
    "Name" = "terraform-vpc"  
  }  
}
```

|               |               |
|---------------|---------------|
| 리소스 타입        | DNS 지원        |
| aws_vpc       | true          |
| 리소스명          | DNS 호스트네임 활성화 |
| terraform_vpc | true          |
| CIDR 블록       | 태그            |
| 10.25.0.0/16  | terraform-vpc |

# 3. Subnet.tf

```
# Public Subnet
resource "aws_subnet" "terraform-pub-subnet-2a" {
  vpc_id = aws_vpc.terraform-vpc.id
  cidr_block = "10.250.1.0/24"
  availability_zone = "ap-northeast-2a"
  map_public_ip_on_launch = "true"
  tags = {
    "Name" = "terraform-pub-subnet-2a"
    "kubernetes.io/role/elb" = "1"
    "kubernetes.io/cluster/terraform-eks-cluster" = "shared"
  }
}

resource "aws_subnet" "terraform-pub-subnet-2c" {
  vpc_id = aws_vpc.terraform-vpc.id
  cidr_block = "10.250.2.0/24"
  availability_zone = "ap-northeast-2c"
  map_public_ip_on_launch = "true"
  tags = {
    "Name" = "terraform-pub-subnet-2c"
    "kubernetes.io/role/elb" = "1"
    "kubernetes.io/cluster/terraform-eks-cluster" = "shared"
  }
}
```

## 리소스 타입

aws\_subnet

## 리소스명

terraform\_pub-subnet-2a

terraform\_pub-subnet-2c

## VPC 연결

terraform-vpc

## CIDR 블록

10.250.1.0/24 -> 서브넷1 IP 범위

10.250.2.0/24 -> 서브넷2 IP 범위

## 가용 영역 (AZ)

ap-northeast-2a / ap-northeast-2c

## Public IP 자동 할당

true -> 생성되는 인스턴스가 자동으로 공인 IP 할당

## 태그

- Name = terraform-pub-subnet-2a -> 관리용 식별 태그
- Name = terraform-pub-subnet-2c -> 관리용 식별 태그
- kubernetes.io/role/elb = 1 -> EKS ALB 연동용
- kubernetes.io/cluster/terraform-eks-cluster = shared -> EKS 클러스터와 연동

# 3. Subnet.tf

```
# Private Subnet
resource "aws_subnet" "terraform-pri-subnet-2a" {
  vpc_id      = aws_vpc.terraform-vpc.id
  cidr_block   = "10.250.11.0/24"
  availability_zone = "ap-northeast-2a"
  tags = {
    "Name" = "terraform-pri-subnet-2a"
    "kubernetes.io/role/internal-elb" = "1"
    "kubernetes.io/cluster/terraform-eks-cluster" = "shared"
  }
}

resource "aws_subnet" "terraform-pri-subnet-2c" {
  vpc_id = aws_vpc.terraform-vpc.id
  cidr_block = "10.250.12.0/24"
  availability_zone = "ap-northeast-2c"
  tags = {
    "Name" = "terraform-pri-subnet-2c"
    "kubernetes.io/role/internal-elb" = "1"
    "kubernetes.io/cluster/terraform-eks-cluster" = "shared"
  }
}
```

## 리소스 타입

aws\_subnet

## 리소스명

terraform\_pri-subnet-2a

terraform\_pri-subnet-2c

## VPC 연결

terraform-vpc

## CIDR 블록

10.250.11.0/24 -> 내부 리소스용1 IP 범위

10.250.12.0/24 -> 내부 리소스용2 IP 범위

## 가용 영역 (AZ)

ap-northeast-2a / ap-northeast-2c

## Public IP 자동 할당

true -> 생성되는 인스턴스가 자동으로 공인 IP 할당

## 태그

- Name = terraform-pri-subnet-2a -> 관리용 식별 태그
- Name = terraform-pri-subnet-2c -> 관리용 식별 태그
- kubernetes.io/role/internal-elb = 1 -> EKS 내부 ELB 연동용
- kubernetes.io/cluster/terraform-eks-cluster = shared -> EKS 클러스터와 연동

## 4. IGW.tf

```
resource "aws_internet_gateway" "terraform-igw" {  
  vpc_id = aws_vpc.terraform-vpc.id  
  tags = {  
    "Name" = "terraform-igw"  
  }  
}
```

### 리소스 타입

aws\_internet\_gateway

### 리소스명

terraform-igw

### 용도

퍼블릭 서브넷 인터넷 접근을 위한 게이트웨이

### 연결

terraform-vpc

### 태그

Name = terraform-igw

# 5. NGW.tf

# 탄력적 IP

```
resource "aws_eip" "terraform-eip" {  
  domain = "vpc"  
}
```

# NAT 게이트웨이

```
resource "aws_nat_gateway" "terraform-ngw" {  
  allocation_id = aws_eip.terraform-eip.id  
  subnet_id     = aws_subnet.terraform-pub-subnet-2a.id  
  tags = {  
    "Name" = "terraform-ngw"  
  }  
}
```

리소스 타입

aws-eip

리소스명

terraform-eip

용도

NAT 게이트웨이 또는  
퍼블릭 인스턴스에 고정 공인 IP 할당

도메인

VPC

리소스 타입

aws\_nat\_gateway

리소스명

terraform-ngw

용도

프라이빗 서비스 인스턴스가  
인터넷 접근 가능하도록 중계

연결 EIP

terraform-eip

퍼블릭 서브넷

terraform-pub-subnet-2a

태그

Name = terraform-ngw



# 6. RT.tf

# 라우팅 테이블

```
resource "aws_route_table" "terraform-pub-rt" {
  vpc_id = aws_vpc.terraform-vpc.id

  tags = {
    "Name" = "terraform-pub-rt"
  }
}

resource "aws_route_table" "terraform-pri-rt" {
  vpc_id = aws_vpc.terraform-vpc.id

  tags = {
    "Name" = "terraform-pri-rt"
  }
}
```

# 라우팅

```
resource "aws_route" "terraform-pub-rt" {
  route_table_id      = aws_route_table.terraform-pub-rt.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id          = aws_internet_gateway.terraform-igw.id
}

resource "aws_route" "terraform-pri-rt" {
  route_table_id      = aws_route_table.terraform-pri-rt.id
  destination_cidr_block = "0.0.0.0/0"
  nat_gateway_id       = aws_nat_gateway.terraform-ngw.id
}
```

이름

terraform-pub-rt

용도

퍼블릭 서브넷용 라우팅 테이블

연결 VPC

terraform-vpc

라우팅

0.0.0.0 → 인터넷 게이트웨이(terraform-igw)

태그

Name = terraform-pub-rt

이름

terraform-pri-rt

용도

프라이빗 서브넷용 라우팅 테이블

연결 VPC

terraform-vpc

라우팅

0.0.0.0 → NAT 게이트웨이(terraform-ngw)

태그

Name = terraform-pri-rt

# 7. Security Group.tf

# Bastion SG

```
resource "aws_security_group" "terraform-sg-bastion" {  
  name      = "terraform-sg-bastion"  
  description = "for Bastion Server"  
  vpc_id    = aws_vpc.terraform-vpc.id  
  tags = {  
    "Name" = "terraform-sg-bastion"  
  }  
}
```

```
ingress {  
  from_port = -1  
  to_port   = -1  
  protocol  = "icmp"  
  cidr_blocks = ["0.0.0.0/0"]  
  description = "for ping"  
}
```

```
ingress {  
  from_port = 22  
  to_port   = 22  
  protocol  = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
  description = "for SSH"  
}
```

```
ingress {  
  from_port = 80  
  to_port   = 80  
  protocol  = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
  description = "for HTTP"  
}
```

```
ingress {  
  from_port = 443  
  to_port   = 443  
  protocol  = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
  description = "for HTTPS"  
}
```

```
ingress {  
  from_port = 3306  
  to_port   = 3306  
  protocol  = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
  description = "for DB"  
}
```

```
egress {  
  from_port = 0  
  to_port   = 0  
  protocol  = "-1"  
  cidr_blocks = ["0.0.0.0/0"]  
}
```

이름

terraform-sg-bastion

용도

Bastion 서버용

Egress

전체 트래픽 허용

허용 규칙

- ICMP (-1) → 전체 (0.0.0.0/0) (Ping 허용)
- TCP 22 (SSH) → 전체
- TCP 80 (HTTP) → 전체
- TCP 443 (HTTPS) → 전체
- TCP 3306 (MySQL DB) → 전체

# 7. Security Group.tf

```
# ALB SG
resource "aws_security_group" "terraform-sg-alb" {
  name = "for ALB"
  vpc_id = aws_vpc.terraform-vpc.id

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "for HTTP"
  }

  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "for HTTPS"
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

이름  
for ALB

용도  
Application Load Balancer용

허용 규칙  
- TCP 80 (HTTP) → 전체  
- TCP 443 (HTTPS) → 전체

Egress  
전체 트래픽 허용 (0.0.0.0/0)



# 7. Security Group.tf

```
# EKS SG
resource "aws_security_group" "terraform-sg-eks-node-
group" {
  name = "for EKS-managed server"
  vpc_id = aws_vpc.terraform-vpc.id

  ingress {
    from_port = -1
    to_port = -1
    protocol = "icmp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "for ping"
  }
}
```

```
ingress {
  from_port = 22
  to_port = 22
  protocol = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
  description = "for SSH"
}
```

```
ingress {
  from_port = 80
  to_port = 80
  protocol = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
  description = "for HTTP"
}
```

```
ingress {
  from_port = 443
  to_port = 443
  protocol = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
  description = "for HTTPS"
}
```

```
egress {
  from_port = 0
  to_port = 0
  protocol = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
```

이름  
for EKS

용도  
EKS용

허용 규칙

- TCP 80 (HTTP) → 전체
- TCP 443 (HTTPS) → 전체

Egress

전체 트래픽 허용 (0.0.0.0/0)

# 7. Security Group.tf

```
# RDS SG
resource "aws_security_group" "terraform-sg-rds" {
  name = "for RDS"
  vpc_id = aws_vpc.terraform-vpc.id

  ingress {
    from_port = 3306
    to_port   = 3306
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "for DB"
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

이름  
for RDS

용도  
RDS(데이터베이스) 용

허용 규칙  
- TCP 3306 (MySQL DB) → 전체

Egress  
전체 트래픽 허용 (0.0.0.0/0)

# 8. Key Pair.tf

```
resource "tls_private_key" "soonge97_key" {
  algorithm = "RSA"
  rsa_bits  = 2048
}

resource "aws_key_pair" "soonge97_aws_key" {
  key_name  = "soonge97"
  public_key = tls_private_key.soonge97_key.public_key_openssh
}

resource "local_file" "soonge97_private_key" {
  content = tls_private_key.soonge97_key.private_key_pem
  filename = "soonge97.pem"
}
```

리소스 타입

tls\_private\_key

리소스명

soonge97\_key

속성

- algorithm: RSA
- rsa\_bits: 2048

추가 설명

Terraform 내부에서 RSA  
개인 키를 생성하는 리소스

리소스 타입

tls\_key\_pair

리소스명

soonge97\_aws\_key

속성

- key\_name: soonge97
- public\_key:  
tls\_private\_key.soonge97\_key.public\_key\_openssh
- filename: soonge97.pem

추가 설명

생성한 공개키를 AWS에 등록해 Key Pair 생성

리소스 타입

local\_file

리소스명

soonge97\_private\_key

속성

- content:  
tls\_private\_key.soonge97\_key.private\_key\_pem

추가 설명

Terraform에서 생성한 개인키를  
로컬 파일로 저장해서 SSH 접속용으로 사용

# 9. EC2.tf

# 1. AWS로부터 최신 Amazon Linux 2023 이미지 ID를 실시간으로 조회

```
data "aws_ami" "bastion_amazon_linux_2023" {
```

```
  most_recent = true
```

```
  owners      = ["amazon"]
```

```
  filter {
```

```
    name = "name"
```

```
    values = ["al2023-ami-2023.*-x86_64"]
```

```
  }
```

```
}
```

# 2. Bastion EC2 생성

```
resource "aws_instance" "terraform-pub-ec2-bastion-2a" {
```

```
  ami = data.aws_ami.bastion_amazon_linux_2023.id # 자동 조회된 최신 ID 매칭
```

```
  instance_type = "t3.micro"
```

```
  vpc_security_group_ids = [aws_security_group.terraform-sg-bastion.id]
```

```
  subnet_id = aws_subnet.terraform-pub-subnet-2a.id
```

```
  key_name = aws_key_pair.soonge97_aws_key.key_name # 리소스 참조로 순서 보장
```

```
  associate_public_ip_address = true
```

```
  root_block_device {
```

```
    volume_size = "8"
```

```
    volume_type = "gp2"
```

```
    tags = {
```

```
      "Name" = "terraform-pub-ec2-bastion-2a"
```

```
    }
```

```
  }
```

```
  tags = {
```

```
    "Name" = "terraform-pub-ec2-bastion-2a"
```

```
  }
```

```
}
```

리소스 타입

aws\_instance

서브넷

terraform-pub-subnet-2a

리소스명

terraform-pub-ec2-bastion-2a

키 페어

soonge97 (SSH 접속용)

AMI

data.aws\_ami.bastion\_amazon\_linux\_2023.id

퍼블릭 IP 할당

true

인스턴스 타입

t3.micro

스토리지

Root Volume: 8GB, gp2 타입

보안 그룹

Name = terraform-sg-bastion

태그

Name = terraform-pub-ec2-bastion-2a

# 10. RDS.tf

```
# RDS 서브넷 그룹 생성
resource "aws_db_subnet_group" "terraform-rds-subnet-group" {
  name      = "terraform-rds-subnet-group"
  subnet_ids = [aws_subnet.terraform-pri-subnet-2a.id, aws_subnet.terraform-
pri-subnet-2c.id]

  tags = {
    Name = "terraform-rds-subnet-group"
  }
}
```

## 리소스 타입

aws\_db\_subnet\_group

## 리소스 명

terraform-rds-subnet-group

## 서브넷

aws\_subnet.terraform-pri-subnet-2a.id

aws\_subnet.terraform-pri-subnet-2c.id

## 태그

NAME = terraform-rds-subnet-group



# 10. RDS.tf

# MariaDB Parameter Group 설정

```
resource "aws_db_parameter_group" "terraform-mariadb-parameter-group" {  
  name      = "terraform-mariadb-parameter-group"  
  family    = "mariadb10.11" # 사용 중인 MariaDB 버전에 맞게 조정  
  description = "Custom parameter group for MariaDB"
```

```
  parameter {  
    name = "max_connections"  
    value = "150"  
  }
```

```
  parameter {  
    name = "time_zone"  
    value = "Asia/Seoul"  
  }
```

```
  parameter {  
    name = "character_set_client"  
    value = "utf8mb4"  
  }
```

```
  parameter {  
    name = "character_set_connection"  
    value = "utf8mb4"  
  }
```

```
  parameter {  
    name = "character_set_database"  
    value = "utf8mb4"  
  }
```

```
  parameter {  
    name = "character_set_filesystem"  
    value = "utf8mb4"  
  }
```

```
  parameter {  
    name = "character_set_results"  
    value = "utf8mb4"  
  }
```

```
  parameter {  
    name = "character_set_server"  
    value = "utf8mb4"  
  }
```

```
  parameter {  
    name = "collation_connection"  
    value = "utf8mb4_unicode_ci"  
  }
```

```
  parameter {  
    name = "collation_server"  
    value = "utf8mb4_unicode_ci"  
  }
```

```
  parameter {  
    name = "binlog_format"  
    value = "ROW"  
  }
```

```
}
```

리소스 타입

aws\_db\_parameter\_group

리소스 명

terraform-mariadb-parameter-group

DB 엔진 패밀리

mariadb10.11

설정 값

- max\_connection = 150 → 동시 접속자 수 조정
- time\_zone = Asia/Seoul → 한국 시간대 설정
- character\_set\_\* = utf8mb4 → 최신 UTF-8 문자셋 적용
- collation\_\* = utf8mb4\_unicode\_ci → 문자열 정렬 규칙
- binlog\_format = ROW → 복제/백업 안정성 강

# 10. RDS.tf

# RDS 생성

```
resource "aws_db_instance" "terraform-mariadb-rds" {
  identifier_prefix = "terraform-mariadb-rds"
  allocated_storage = 10
  engine           = "mariadb"
  engine_version   = "10.11"
  instance_class    = "db.t3.micro"
  db_name           = "terraform-mariadb-rds" # 영문자로 시작해야 하며 영문자와 숫자, _만 가능
  username          = "root"
  password          = "Password1234" # 8자 이상, 영문 대소문자, 숫자 및 특수 문자를 혼합하여 사용
  parameter_group_name = "terraform-mariadb-parameter-group"
  skip_final_snapshot = true
  #multi_az          = true
  db_subnet_group_name = aws_db_subnet_group.terraform-rds-subnet-group.name
  vpc_security_group_ids = [aws_security_group.terraform-sg-rds.id]

  tags = {
    Name = "terraform-mariadb-rds"
  }
}
```

DB명

terraform-mariadb-rds

계정 정보

- username = root
- password = Password1234

파라미터 그룹

terraform-mariadb-parameter-group

네트워크

- 서브넷 그룹: terraform-rds-subnet-group
- 보안 그룹: terraform-sg-rds

기타설정

- skip\_final\_snapshot = true  
(삭제 시 스냅샷 미생성 - 개발환경용)
- multi\_az → 주석처리  
(실무에서는 활성화 권장)

리소스 타입

aws\_db\_instance

리소스명

terraform-mariadb-rds

DB 엔진

mariadb (10.11.8)

인스턴스 클래스

db.t3.micro

스토리지

10GB

# 11. S3.tf

```
resource "aws_s3_bucket" "mjc-2026-demo" {  
  bucket = "mjct-2026-sjs"  
}
```

리소스 타입

aws\_s3\_bucket

버킷명

mjct-2026-sjs



# 12. ACM.tf

```
# 인증서 생성
resource "aws_acm_certificate" "cert" {
  domain_name      = "*.jaeseop.store"
  validation_method = "DNS"

  tags = {
    Name = "jaeseop.store"
  }

  lifecycle {
    create_before_destroy = true
  }
}

# 인증서 검증
resource "aws_route53_record" "route53_ssl" {
  for_each = {
    for dvo in aws_acm_certificate.cert.domain_validation_options :
    dvo.domain_name => {
      name    = dvo.resource_record_name
      record  = dvo.resource_record_value
      type    = dvo.resource_record_type
    }
  }

  allow_overwrite = true
  name            = each.value.name
  records         = [each.value.record]
  ttl             = 60
  type            = each.value.type
  zone_id         = aws_route53_zone.route53.zone_id
}
```

## 리소스 타입

aws\_acm\_certificate

## 리소스명

cert

## 생성 이름

.jaeseop.store

## 서브넷

DNS

## 퍼블릭 여부

create\_before\_destroy = true

## 태그

Name = jaeseop.store

## 리소스 타입

aws\_route53\_record

## 리소스명

route53\_ssl

## 구성 방식

- for\_each 구문을 활용하여, aws\_acm\_certificate.cert.domain\_validation\_options 값들을 자동으로 가져옴
- ACM이 제시한 검증 레코드(CNAME)를 Route53에 자동 등록

## 특징

- allow\_overwrite = true  
→ 기존 레코드가 있더라도 덮어쓰기 가능
- ttl = 60 → 빠른 검증 반영
- zone\_id = aws\_route53\_zone.route53.zone\_id  
→ 앞서 만든 Hosted Zone에 연결

# 13. Route53.tf

```
resource "aws_route53_zone" "route53" {  
  name = "jaeseop.store"  
}
```

## 리소스 타입

aws\_route53\_zone (AWS Route53에 호스팅 존 생성)

## 도메인명

jaeseop.store

## 리소스명

route53

# 14. ALB.tf

##### lb 생성

```
resource "aws_lb" "web-lb" {
  name          = "web-lb"
  subnets      = [aws_subnet.terraform-pub-subnet-2a.id,
aws_subnet.terraform-pub-subnet-2c.id]
  internal      = false
  load_balancer_type = "application"
  tags = {
    "Name" = "web-lb"
  }

  # ALB 전용 보안 그룹으로 올바르게 매칭되어 있습니다.
  security_groups = [aws_security_group.terraform-sg-alb.id]
}
```

리소스 타입

aws\_lb

퍼블릭 여부

false

리소스명

web-lb

보안 그룹

aws\_security\_group.terraform-sg-alb

생성 이름

web-lb

타입

application

서브넷

terraform-pub-subnet-2a

terraform-pub-subnet-2c

태그

Name = web-lb

# 14. ALB.tf

##### Target Group

```
resource "aws_lb_target_group" "terraform-prd-tg" {
```

```
  name = "terraform-prd-tg"
```

```
  port = 80
```

```
  protocol = "HTTP"
```

```
  vpc_id = aws_vpc.terraform-vpc.id
```

# [수정] Nginx 웹 서버가 HTML 파일을  
확실하게 응답하는지 체크하도록 경로를 명시했습니다.

```
  health_check {
```

```
    port = 80
```

```
    path = "/index.html"
```

```
  }
```

```
  tags = {
```

```
    "Name" = "terraform-prd-tg"
```

```
  }
```

```
}
```

##### Listener

```
resource "aws_lb_listener" "terraform-prd-listener" {
```

```
  load_balancer_arn = aws_lb.web-lb.arn
```

```
  port = "80"
```

```
  protocol = "HTTP"
```

```
  default_action {
```

```
    target_group_arn = aws_lb_target_group.terraform-prd-tg.arn
```

```
    type = "forward"
```

```
  }
```

```
}
```

리소스 타입

aws\_lb\_target\_group

리소스명

terraform-prd-tg

트래픽 규칙

port = 80 / protocol = "HTTP"

VPC

aws\_vpc.terraform-vpc.id

헬스 체크

port = 80

path = "/index.html"

태그

Name = terraform-prd-tg

리소스 타입

aws\_lb\_listener

리소스명

terraform-prd-listener

로드밸런서 ARN

aws\_lb.web-lb.arn

Default Auction

Target Group(terraform-prd-tg)으로 트래픽 전달

# 15. AutoScaling.tf

```
# [자동화] AWS로부터 현재 사용 가능한
최신 Amazon Linux 2023 이미지 ID를 실시간으로 조회
data "aws_ami" "amazon_linux_2023" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-2023.*-x86_64"]
  }
}
```

# 1. Launch Template 생성

```
resource "aws_launch_template" "as_template" {
  # 캐시 고임을 완전히 파괴하기 위해 이를 접두사를 신규 버전으로 변경
  name_prefix = "terraform-lt-v3-"

  # 실시간으로 찾아온 최신 살아있는 ID를 자동으로 삽입
  image_id = data.aws_ami.amazon_linux_2023.id
  instance_type = "t3.micro" # 서울 리전 가용 영역 자원 부족 우회용 규격
  key_name = aws_key_pair.soonge97_aws_key.key_name
  vpc_security_group_ids = [aws_security_group.terraform-sg-bastion.id]
```

```
# 조장이 작성한 4RSENAL 팀 정보 HTML을 완벽히 이식
user_data = base64encode(<<EOF
#!/bin/bash
sudo dnf update -y
sudo dnf install -y nginx

# Nginx 서비스를 먼저 활성화하고 켵니다.
sudo systemctl enable nginx
sudo systemctl start nginx

# 기존 기본 화면을 지우고 내가 만든 HTML을 넣습니다.
sudo rm -f /usr/share/nginx/html/index.html
sudo cat << 'HTML' > /usr/share/nginx/html/index.html

<!DOCTYPE html>

[HTML 내용 생략]
```

리소스 타입

aws\_launch\_template

리소스명

as\_template

접두사

terraform-lt-v3-

이미지

data.aws\_ami.amazon\_linux\_2023.id

```
# 변경된 HTML을 적용하기 위해 Nginx를 안전하게 리로드
sudo systemctl reload nginx
EOF
)
tag_specifications {
  resource_type = "instance"
  tags = {
    Name = "jeff-userdata"
  }
}
lifecycle {
  create_before_destroy = true
}
```

인스턴스 타입

t3.micro

키페어

soonge97

보안 그룹

aws\_security\_group.terraform-sg-bastion.id

User Data

패키지 업데이트

태그

Name = "jeff-userdata"

라이프사이클 정책

create\_before\_destroy = true

(수정 시 기존 리소스 삭제



# 15. AutoScaling.tf

# 2. Auto-Scaling 그룹 생성

```
resource "aws_autoscaling_group" "terraform-prd-asg" {
  name                = "terraform-prd-asg"
  vpc_zone_identifier = [aws_subnet.terraform-pub-subnet-2a.id, aws_subnet.terraform-pub-subnet-2c.id]
  min_size            = 2
  max_size            = 4
  desired_capacity    = 3
  health_check_grace_period = 120
  health_check_type   = "ELB"
  target_group_arns   = [aws_lb_target_group.terraform-prd-tg.arn]

  launch_template {
    id      = aws_launch_template.as_template.id
    version = "$Latest"
  }
  depends_on = [
    aws_lb.web-lb,
    aws_lb_target_group.terraform-prd-tg
  ]
  lifecycle {
    create_before_destroy = true
  }
}
```

리소스 타입

aws\_autoscaling\_group

리소스명

terraform-pri-asg

생성 이름

terraform-pri-asg

VCP 서브넷

terraform-pub-subnet-2a

terraform-pub-subnet-2c

스케일링 설정

인스턴스 수: 2-4

기본 용량: 3

헬스 체크

유형: ELB

유예 기간: 120s

대상 그룹 연결

aws\_lb\_target\_group.terraform-prd-tg.arn

라이프사이클 정책

create\_before\_destroy = true

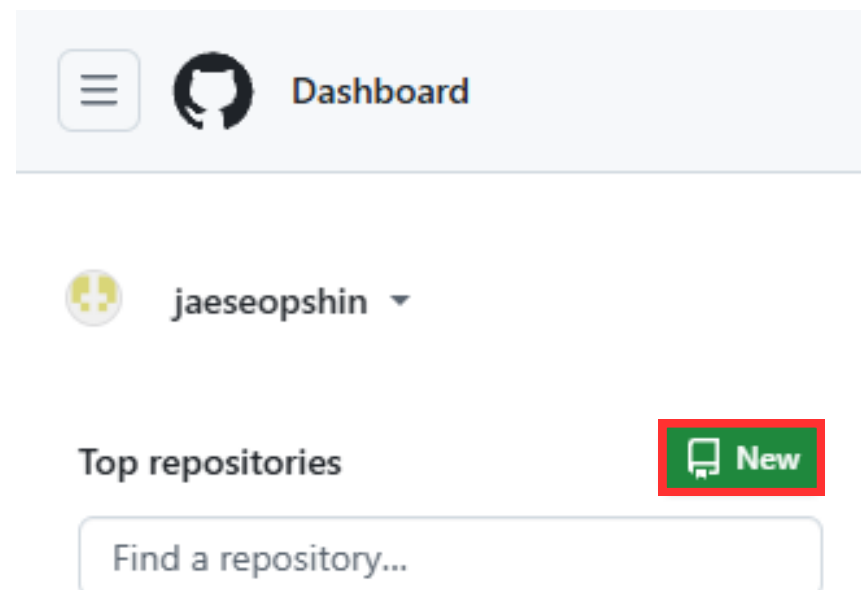
# 6. Github, Terraform Cloud, AWS 연동

---

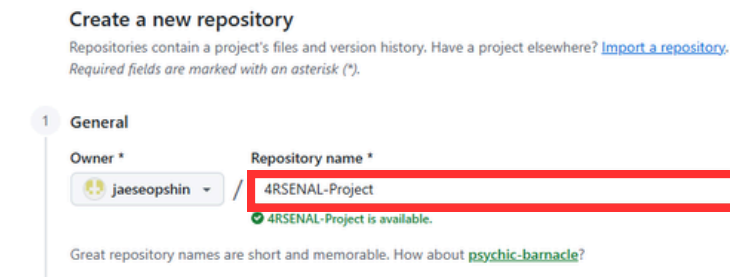


# 6-1. Github Repository 생성

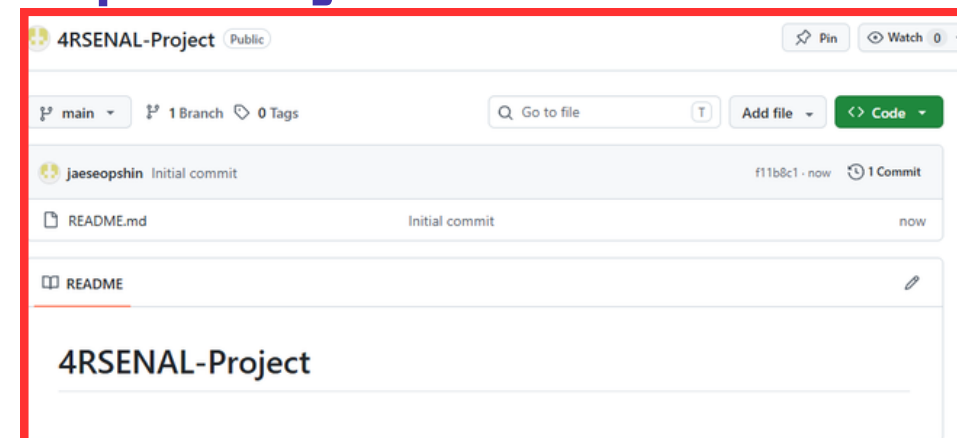
## Github Repository 생성



## Repository 이름 설정



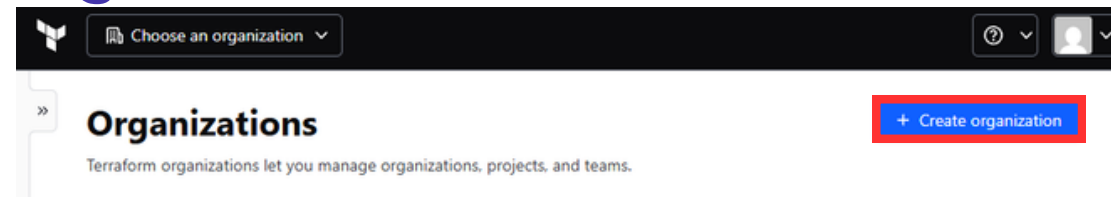
## Repository 생성 확인





# 6-2. Terraform Cloud 초기 구성 & Github 연동

## Organization 생성



## 이름 / E-mail 추가

The screenshot shows the 'Create organization' form. The 'Terraform organization name' field is highlighted with a red box, containing the text '4RSENAL-Project'. The 'Email address' field is also highlighted with a red box, containing the text 'powerlab123@naver.com'. Below the fields are 'Create organization' and 'Cancel' buttons.

## Github와 연동할 Workspace 생성

### Create a new Workspace

HCP Terraform organizes your infrastructure resources by w  
[workspaces in HCP Terraform.](#)

### Choose your workflow

#### Version Control Workflow

Trigger runs based on changes to configuration in repositories.

Best for those who need traceability and transparency

## Github 선택

### Connect to a version control provider

Choose the version control provider that hosts the Terraform configuration for this workspace.

Project: Default Project

Public

GitHub

GitHub App

[Connect to a different VCS](#)

## Github Repository 선택

### Configure Settings

#### Workspace Name

4RSENAL-Project

The name of your workspace is unique and used in tools, routing, and UI. Dashes, underscores, and alphanumeric characters are permitted. Learn more about [naming workspaces](#).

#### Description (Optional)

Workspace description

#### Tags

Use tags to correlate, organize, or filter your resources using single values or key/value pairs.

+ Add tag

# 6-3. Terraform Cloud & AWS 연동

## Variable 생성

4RSENAL-Project / Workspaces / 4RSENAL-Project / Overview

### 4RSENAL-Project

ID: ws-nodkfj27NKB17QeuC

[Add workspace description](#)

Unlocked Resources 0 Tags 0 Terraform v1.15.4 Updated a minute ago

Configuration uploaded successfully

#### Next step: configure variables

Configure any required variables (such as access keys or configuration values) before starting a run.

[Configure variables](#)

## ACCESS KEY 입력

Key

AWS\_ACCESS\_KEY\_ID

Value

value

☐ HCL ☐ Sensitive

Description (Optional)

description (optional)

Key

AWS\_SECRET\_ACCESS\_KEY

Value

value

☐ HCL ☐ Sensitive

Description (Optional)

description (optional)

## 자동 Apply 활성화

### Auto-apply

Sets this workspace to automatically apply changes for successful runs. If disabled, runs require operator approval.

☒ Auto-apply API, UI, & VCS runs

Automatically apply changes when a Terraform plan is successful. If this workspace is linked to version control, a push to the default branch of the linked repository will trigger a plan and apply.

☒ Auto-apply run triggers

Run triggers create new plans whenever a specified source workspace completes an apply. This setting automatically applies these automatically created runs.

### Terraform version

1.15.4

The Terraform version used for runs in this workspace. By default, HCP Terraform uses the latest Terraform version. You can alternatively select an exact version or use a [version constraint](#).

### Terraform working directory

The directory that Terraform will execute within. This defaults to the root of your repository and is typically set to a subdirectory matching the environment when multiple environments exist within the same repository.

### Remote state sharing

Choose whether this workspace should share state with the entire organization, or only with specific approved workspaces. The `terraform_remote_state` data source relies on state sharing to access workspace outputs.

☐ Share with all workspaces in this organization

☐ Share with all workspaces in this project

☒ Share with specific workspaces

Select workspaces

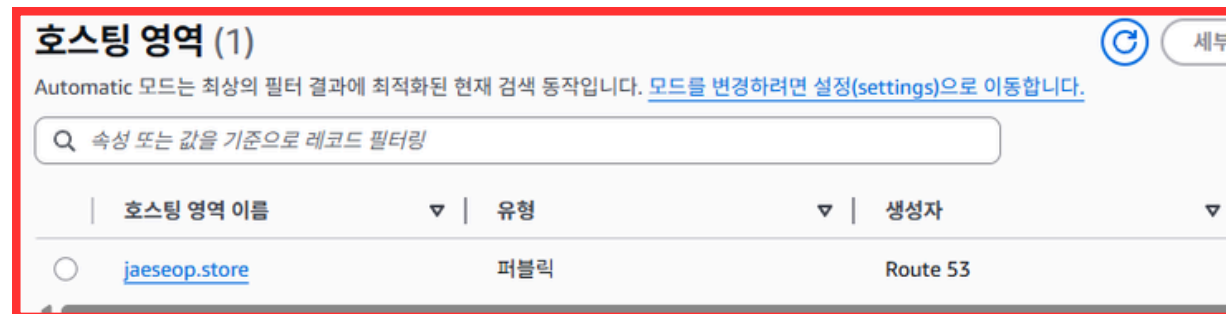
# 7. Route 53 도메인 등록

---

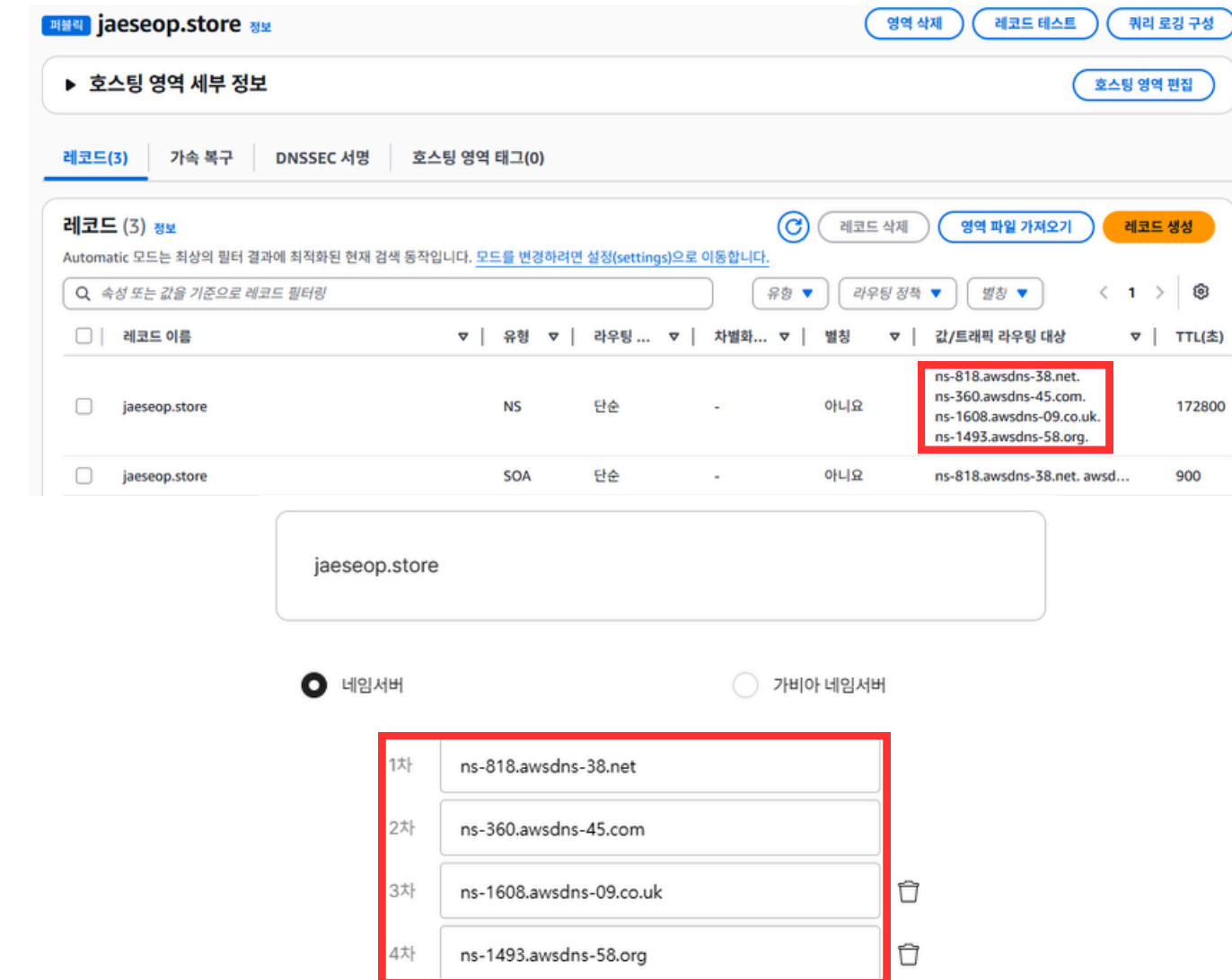


# 7-1. 호스트 영역 확인 7-2. NS 레코드 확인

## 호스팅 영역 확인

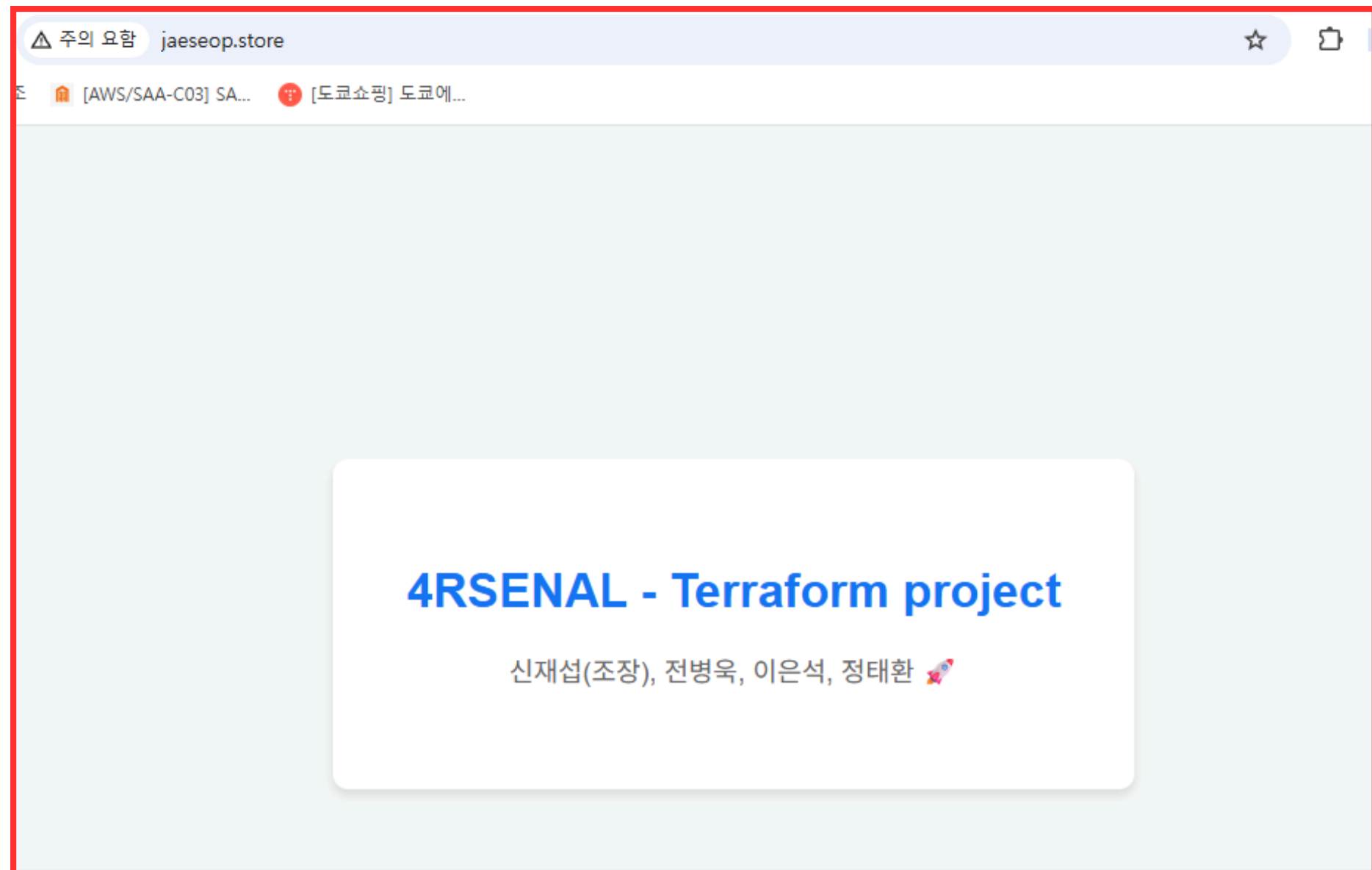


## NS 레코드 확인 및 입력



## 7-3. 출력 결과 확인

jaeseop.store 입력 후  
출력 결과 확인





**THANK YOU**

**감사합니다**